INFO 5 – Projet 2 : Cryptographie élémentaire

Niveau : L1 Maths | Séance 2 sur 4 (3h) : Attaque par Force Brute et Filtrage

Objectifs de la séance

- Comprendre la notion d'espace des clés.
- Implémenter une attaque par recherche exhaustive (tester toutes les solutions possibles).
- Utiliser des boucles pour générer des dizaines de solutions.
- Automatiser la détection avec une recherche de mots-clés, puis l'améliorer avec un **système de compteur** pour éviter les faux positifs.

Partie 1 : L'attaque par Force Brute (Naïve)

L'espace des clés du chiffrement de César est très petit : il n'y a que 26 décalages possibles (de 0 à 25). Un ordinateur peut donc générer instantanément toutes les combinaisons. C'est ce qu'on appelle une attaque par **force brute** ou recherche exhaustive.

Exercice 1 : Afficher toutes les combinaisons

Écrivez un script qui prend un texte chiffré court et affiche les 26 déchiffrements possibles dans la console. L'utilisateur (vous) devra lire les 26 lignes pour repérer visuellement la phrase qui a du sens.

```
def attaque_force_brute(texte_chiffre):
    """Affiche toutes les tentatives de dechiffrement dans la console."""
```

Testez votre fonction avec un petit message chiffré dans la console pour vérifier que vous voyez bien apparaître le message en clair à la bonne ligne.

Partie 2 : Le Filtrage Automatique (Recherche par dictionnaire)

Lire 26 lignes est faisable pour un humain. Mais si nous utilisions un algorithme avec des millions de clés, ce serait impossible. Nous allons donc apprendre à l'ordinateur à **reconnaître du français** tout seul, en cherchant des mots très courants (des "mots de liaison").

Exercice 2a : L'algorithme de détection simple

Écrivez une fonction qui ne renvoie **que** le texte déchiffré contenant au moins un des mots très fréquents du français.

Attention : Pensez à inclure les espaces autour des mots dans votre liste (ex : " le " et non "le") pour éviter de détecter "le" à l'intérieur du mot "poule".

```
def attaque_par_dictionnaire(texte_chiffre):
    """Trouve la cle en cherchant un seul mot du dictionnaire."""
    mots_frequents = [" le ", " de ", " et ", " un ", " est ", " que "]

                return tentative # On s'arrete des le premier mot trouve

    return "Echec : aucun mot du dictionnaire n'a ete trouve."
```

Le problème des "Faux Positifs" :
Testez votre fonction `attaque_par_dictionnaire` avec ce message chiffré :
`message = "hx ij axh rt bthhpvt at igthdg thi jc extvt"`

Vous remarquerez que votre algorithme s'arrête en affichant du charabia ! Que s'est-il passé ? Un mauvais décalage a généré par hasard la suite de lettres " de " dans un mot sans aucun sens. Notre fonction s'est arrêtée au mauvais endroit. C'est ce qu'on appelle un **faux positif**.

Exercice 2b : Amélioration avec un système de Compteur

Pour rendre notre attaque plus robuste, copiez votre fonction précédente et renommez-la `attaque_par_dictionnaire_amelioree`. Modifiez-la pour intégrer un **compteur** (ou *score*). On ne validera le texte que si on y trouve au moins **deux** mots fréquents différents. Testez-la à nouveau sur le `message` pour vérifier qu'elle trouve enfin la bonne phrase !

```
def attaque_par_dictionnaire_amelioree(texte_chiffre):
    """Trouve la bonne cle en exigeant au moins 2 mots reconnus."""
    mots_frequents = [" le ", " de ", " et ", " un ", " est ", " que "]



        # On verifie si notre seuil de confiance est atteint


    return "Echec : texte non reconnu ou trop court pour atteindre le seuil."
```

Partie 3 : Casser un vrai fichier (Le test final)

Mettez-vous dans la peau d'un attaquant qui a intercepté le fichier `message_chiffre.txt`. Vous ne connaissez pas la clé qui a été utilisée pour le chiffrer.

Exercice 3 : Le script d'interception

Utilisez vos fonctions de lecture de fichier et votre attaque **améliorée** pour casser le code, puis sauvegardez le résultat en clair dans un nouveau fichier.

```
# --- Script principal de l'attaquant ---

# 1. Lire le fichier intercepte
texte_intercepte = lire_fichier("message_chiffre.txt")

# 2. Lancer l'attaque intelligente
texte_casse = attaque_par_dictionnaire_amelioree(________)

# 3. Sauvegarder la decouverte si l'attaque a reussi
if "Echec" not in texte_casse:
    ecrire_fichier("message_casse.txt", texte_casse)
    print("Le fichier a ete casse et sauvegarde avec succes !")
else:
    print("L'attaque a echoue.")
```

Partie 4 : Les limites de cette attaque (Bonus réflexion)

Exercice 4 : Les failles du dictionnaire

Créez un court message secret, chiffrez-le, et essayez de le casser avec votre `attaque_par_dictionnaire_amelioree`. Trouvez deux situations distinctes (deux types de messages différents) où cette méthode informatique **échoue** à trouver la bonne clé, bien qu'il s'agisse d'un chiffrement de César valide.

Indice : Que se passe-t-il si le message est très court par rapport à notre seuil ? Que se passe-t-il s'il s'agit d'une liste de noms propres ? Nous verrons comment résoudre ce problème mathématiquement avec l'analyse fréquentielle à la Séance 3 !